```
import pandas as pd
```

```
df= pd.read_csv("/content/Salary_Data.csv")
```

```
df.head()
```

	Age	Gender	Education Level	Job Title	Years of Experience	Salary
0	32.0	Male	Bachelor's	Software Engineer	5.0	90000.0
1	28.0	Female	Master's	Data Analyst	3.0	65000.0
2	45.0	Male	PhD	Senior Manager	15.0	150000.0
3	36.0	Female	Bachelor's	Sales Associate	7.0	60000.0
4	52.0	Male	Master's	Director	20.0	200000.0

Next steps: Generate code with df | New interactive sheet

```
df.info()
```

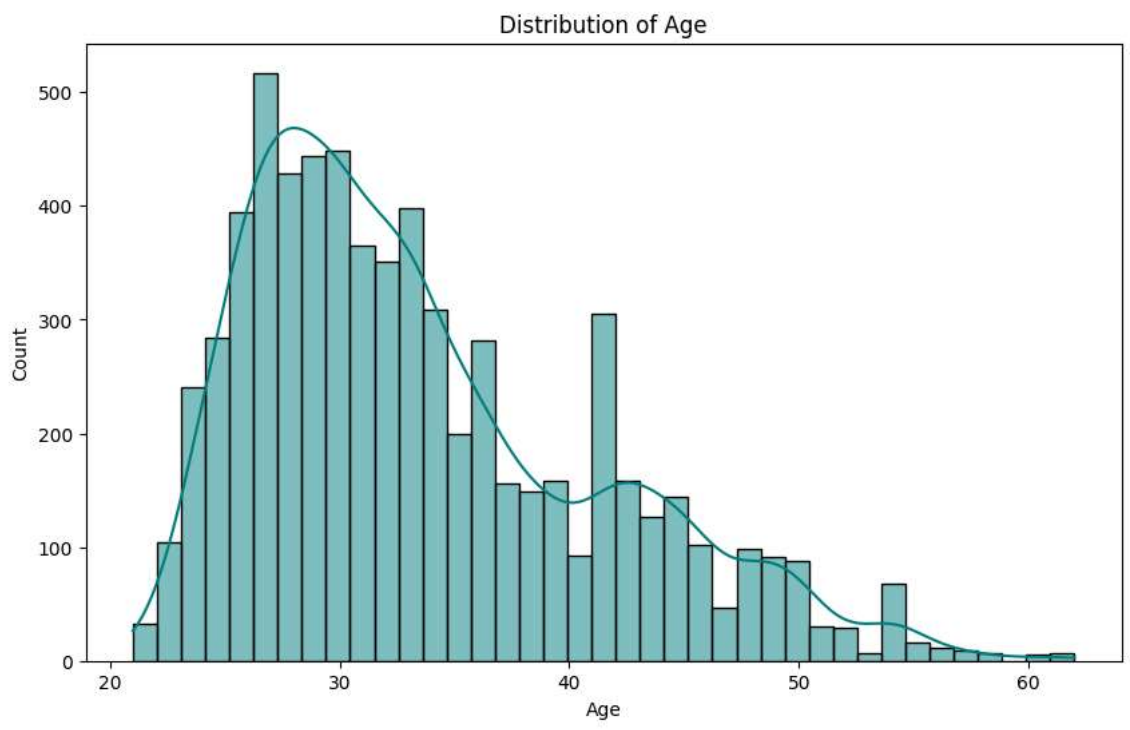
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6704 entries, 0 to 6703
Data columns (total 6 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Age                  6702 non-null   float64
 1   Gender               6702 non-null   object
 2   Education Level      6701 non-null   object
 3   Job Title            6702 non-null   object
 4   Years of Experience  6701 non-null   float64
 5   Salary               6699 non-null   float64
dtypes: float64(3), object(3)
memory usage: 314.4+ KB
```

```
df.isna().sum()
```

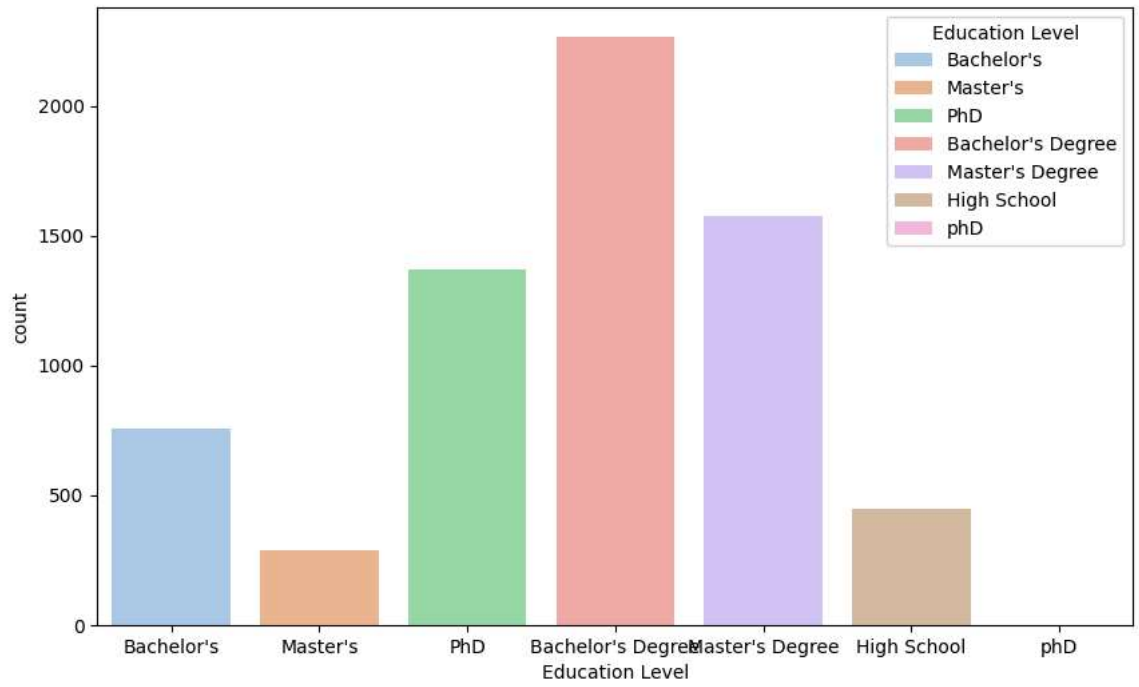
	0
Age	2
Gender	2
Education Level	3
Job Title	2
Years of Experience	3
Salary	5

dtype: int64

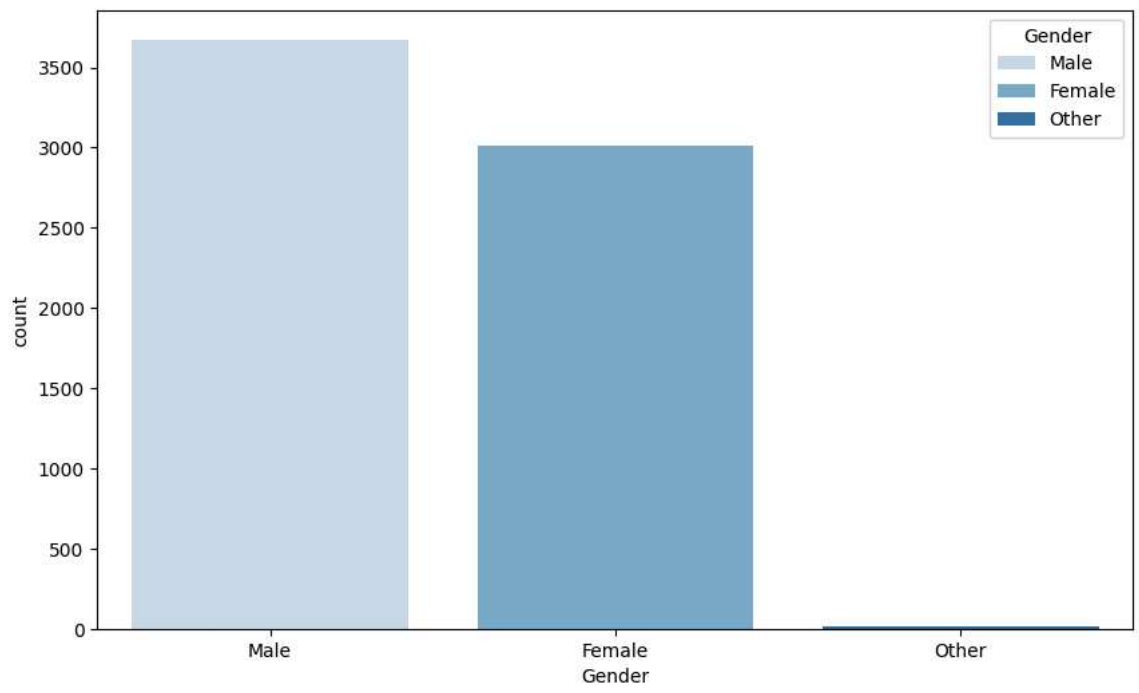
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,6))

sns.histplot(data = df, x = "Age", color ="teal" ,kde=True)
plt.title("Distribution of Age")
plt.show()
```



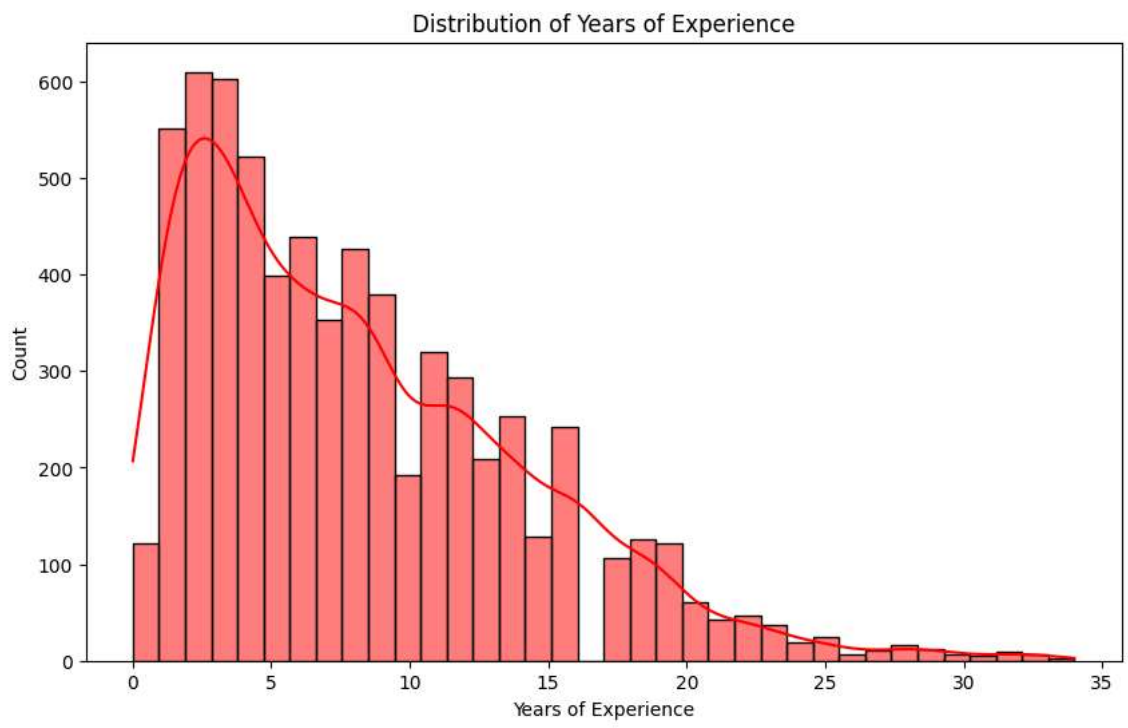

```
plt.figure(figsize=(10,6))

sns.countplot(data = df, x ="Education Level", hue= "Education Level", palette = "pastel")

plt.show()
```




```
plt.figure(figsize=(10,6))

sns.countplot(data = df, x ="Gender" , hue = "Gender" , palette ="Blues")

plt.show()
```




```
plt.figure(figsize=(10,6))

sns.histplot(data = df, x = "Years of Experience", color = "Red", kde = True)
plt.title("Distribution of Years of Experience")
plt.show()
```




```
plt.figure(figsize=(10,6))

sns.histplot(data = df, x = "Salary" , color = "Magenta", kde =True)
plt.title("Distribution of Salary")
plt.show()
```


```
df.describe()
```

	Age	Years of Experience	Salary
count	6702.000000	6701.000000	6699.000000
mean	33.620859	8.094687	115326.964771
std	7.614633	6.059003	52786.183911
min	21.000000	0.000000	350.000000
25%	28.000000	3.000000	70000.000000
50%	32.000000	7.000000	115000.000000
75%	38.000000	12.000000	160000.000000
max	62.000000	34.000000	250000.000000

```
df["Gender"].unique()
```

```
array(['Male', 'Female', nan, 'Other'], dtype=object)
```

```
df["Education Level"].unique()
```

```
array(["Bachelor's", "Master's", 'PhD', nan, "Bachelor's Degree",
       "Master's Degree", 'High School', 'phD'], dtype=object)
```

```
df["Education Level"] = df["Education Level"].replace({"Bachelor's Degree":"Bachelor's" ,"Master's Degree" :"Master's" , "phD" : "PhD"})
```

```
df["Education Level"].unique()
```

```
array(["Bachelor's", "Master's", 'PhD', nan, 'High School'], dtype=object)
```

```
df["Job Title"].unique()
```

```
       'Senior Project Manager', 'Principal Scientist',
       'Supply Chain Manager', 'Senior Marketing Manager',
       'Training Specialist', 'Research Scientist',
       'Junior Software Developer', 'Public Relations Manager',
       'Operations Analyst', 'Product Marketing Manager',
       'Senior HR Manager', 'Junior Web Developer',
       'Senior Project Coordinator', 'Chief Data Officer',
       'Digital Content Producer', 'IT Support Specialist',
       'Senior Marketing Analyst', 'Customer Success Manager',
       'Senior Graphic Designer', 'Software Project Manager',
       'Supply Chain Analyst', 'Senior Business Analyst',
       'Junior Marketing Analyst', 'Office Manager', 'Principal Engineer',
       'Junior HR Generalist', 'Senior Product Manager',
       'Junior Operations Analyst', 'Senior HR Generalist',
       'Sales Operations Manager', 'Senior Software Developer',
       'Junior Web Designer', 'Senior Training Specialist',
       'Senior Research Scientist', 'Junior Sales Representative',
       'Junior Marketing Manager', 'Junior Data Analyst',
       'Senior Product Marketing Manager', 'Junior Business Analyst',
       'Senior Sales Manager', 'Junior Marketing Specialist',
       'Junior Project Manager', 'Senior Accountant', 'Director of Sales',
       'Junior Recruiter', 'Senior Business Development Manager',
       'Senior Product Designer', 'Junior Customer Support Specialist',
       'Senior IT Support Specialist', 'Junior Financial Analyst',
       'Senior Operations Manager', 'Director of Human Resources',
       'Junior Software Engineer', 'Senior Sales Representative',
       'Director of Product Management', 'Junior Copywriter',
```

```
'Digital Marketing Specialist', 'Receptionist',
'Marketing Director', 'Social M', 'Social Media Man',
'Delivery Driver'], dtype=object)
```

```
df["Job Title"].nunique()
```

```
193
```

```
df.dropna(inplace =True)
```

```
X = df.drop(columns = "Salary" )
y = df["Salary"]
```

```
X
```

	Age	Gender	Education Level	Job Title	Years of Experience
0	32.0	Male	Bachelor's	Software Engineer	5.0
1	28.0	Female	Master's	Data Analyst	3.0
2	45.0	Male	PhD	Senior Manager	15.0
3	36.0	Female	Bachelor's	Sales Associate	7.0
4	52.0	Male	Master's	Director	20.0
...	...	...	...	...	...
6699	49.0	Female	PhD	Director of Marketing	20.0
6700	32.0	Male	High School	Sales Associate	3.0
6701	30.0	Female	Bachelor's	Financial Manager	4.0
6702	46.0	Male	Master's	Marketing Manager	14.0
6703	26.0	Female	High School	Sales Executive	1.0

6698 rows × 5 columns

Next steps: Generate code with X | New interactive sheet

```
from sklearn.model_selection import train_test_split

X_train , X_test , y_train , y_test = train_test_split(X,y , test_size = 0.2 , random_state = 42)
```

```
from sklearn.preprocessing import StandardScaler , OneHotEncoder , LabelEncoder
```

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

```
cat_col = ["Gender","Education Level" , "Job Title"]

num_col = ["Age","Years of Experience"]
```

```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_col),
        ('cat', OneHotEncoder(handle_unknown='ignore'), cat_col)
    ])
```

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model_pipeline = Pipeline(steps=[('preprocessor', preprocessor),
                                 ('regressor', model)])

model_pipeline.fit(X_train, y_train)

plt.show()
```

```
y_pred_pipeline = model_pipeline.predict(X_test)
```